**Computer Networks Lab**

**Lab Assignment - 8 & 9**

**Traffic Applications and Trace File**

**<u>Objective:</u> To send some traffic/data in the network topologies created and reading the trace file.**

## Network Agents

Traffic generation in NS-2 is based on the objects of two classes, the class Agent and the class Application. Every node in the network that needs to send or receive traffic has to have an agent attached to it. On top of an agent runs an application. The application determines the kind of traffic that is simulated.

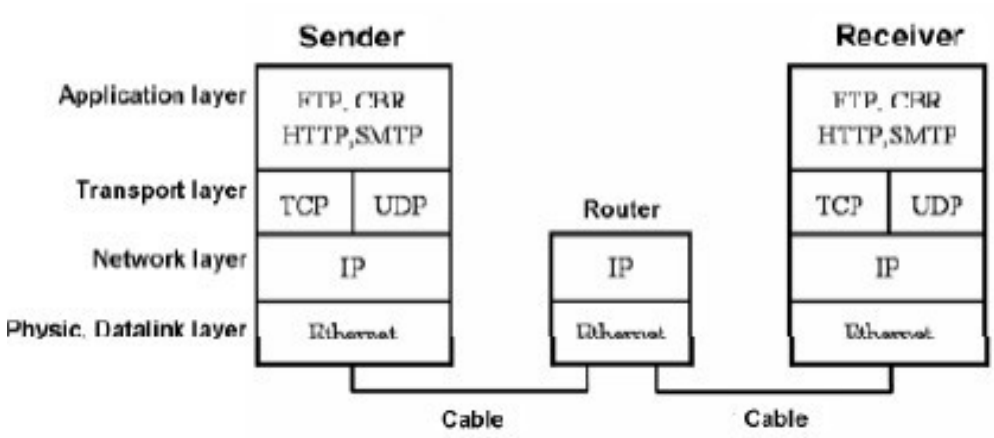



Figure 1: The Realistic Network

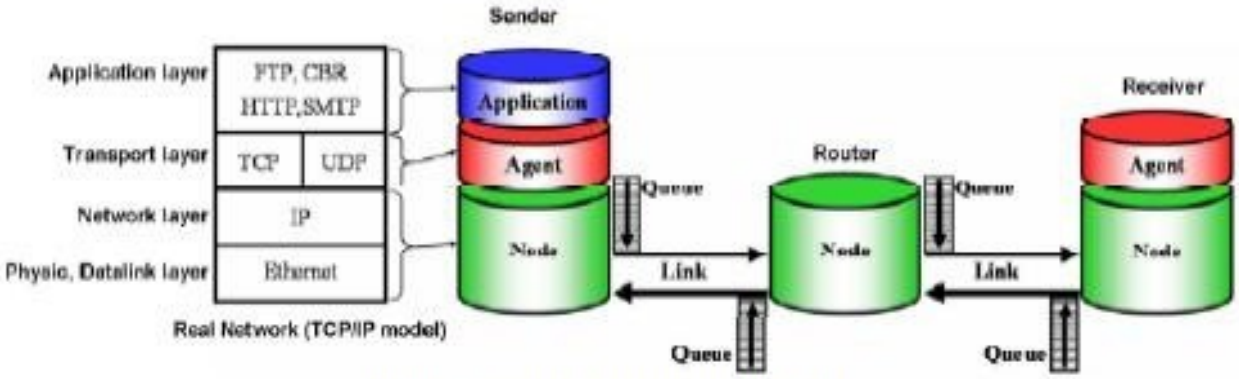



Figure 2: The Network inside NS2

Figure 2: The Network inside NS2

The node (terminal, router) in real network have 4 layer using TCP/IP model as shown in Fig.1. Actually, forwarding router have only 2 bottom layer. In the otherwise, 4 layers of TCP/IP model in NS2 are shown in Fig. 2. Two bottom layers are ready automatically when a node setting up. **TCP or UDP** in transport layer are shown as **Agent. FTP, CBR** are in **application layers**. Actually, the sender will set the application for creating data while the receive will not because it is no need to simulate the receiving data in application layer.

**The following is steps to creating a simple traffic scenario file. Step0: Declare Simulator and setting output file**
**Step1: Setting Node and Link**
**Step2: Setting Agent**
**Step3: Setting Application**
**Step4: Setting Simulation time and schedules**
**Step5: Declare finish.**

## Write Scenario File:

### Step0: Declare Simulator and setting output file

Declare simulator and setting output file is written as below.

```
set ns [new Simulator]
set file [open out.tr w]
$ns trace-all $file
set namfile [open out.nam w]
$ns namtrace-all $namfile
```

### Step1: Setting Node and Link

Setting a node is shown as below.

```
set n0 [$ns node]
```

then the node which have name as n0 is ready to use. (in Tcl file it will refer to $n0). Node is numbered from 0. Link is declared as below.

```
$ns duplex-link $n0 $n2 3Mb 5ms DropTail
$ns duplex-link-op $n0 $n2 orient right-down
```

In line 1, the link between two nodes n0 and n2 has bandwidth 3Mbps and delay 5ms. A biconnection link between n0 and n2 is declared. DropTail is a waiting queue type, if the queue overflow then the new entered packet will be dropped, similar to buffer of general realistic network. Line 2 setting position of node and link for Nam. It does not affect to the result of simulation.

The length of queue is set as below

```
$ns queue-limit $n2 $n3 20
$ns duplex-link-op $n2 $n3 queuePos 0.5
```

In line 1, the length of Queue on the link from n2 to n3 is 20[packets]. In line 2, the position of Queue is set for executing simulation result on Nam, 0.5 is the angle between link and queue, it equal to (0.5π).

**Step2: Setting**

**Agent UDP Agent:**

Using UDP for simulation, the sender set the Agent as UDP Agent while the receiver set to Null Agent. Null Agents do nothing except receiving the packet. Setting of Agent as flowing.

```
set udp [new Agent/UDP]
$ns attach-agent $n0 $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
$udp set fid_ 0
$ns color 0 blue
```

In first 4 lines, udp, null Agent is set for n0, n3. Line 5 declared the transmission between udp and null. Line 6 set the number for data flow of udp. This number will be recorded to all packet which are sent from udp. By using this number, we can easily observer the flow that packet is belong to by looking up trace file. Similarly, in line 7, we mark the color to discrete packet for showing result on Nam.

**TCP Agent:**

Using TCP for simulation, the sender set the Agent as TCP Agent while the receiver set to TCPSink Agent. When receiving a packet, TCPSink Agent will reply an acknowledgment packet (ACK). Setting Agent for TCP is similar to UDP.

```
set tcp [new Agent/TCP]
$ns attach-agent $n1 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
$ns color 1 red
```

Confirm that fid and color is differ from udp.

**Step3: Setting Application**

In general, UDP Agent use CBR Application while TCP Agent use FTP Application.

```
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
set ftp [new Application/FTP]
$ftp attach-agent $tcp
```

**Step4: Setting time schedule for simulation**

Time schedule of a simulation is setting as below:

```
$ns at 1.0 "$cbr start"
$ns at 1.5 "$ftp start"
$ns at 3.0 "$ftp stop"
$ns at 3.5 "$cbr stop"
$ns at 4.0 "finish"
proc finish {} {
        global ns nf
        $ns flush-trace
        #Close the NAM trace file
        close $nf
        #Execute NAM on the trace file
        exec nam out.nam &
        exit 0
}
```

cbr transmit data from 1.0[sec] to 3.5[sec] and ftp transmit data from 1.5[sec] to 3.0[sec].
The finish function is use for output data file which always at the end of simulation.

**Step5: Declare finish**

After finish setting, declare finish is written at the end of file.

```
$ns run
```

**More on UDP Traffic Applications:**

Here, we will discuss four traffic applications that go on top of a UDP agent to simulate network traffic.

1. **CBR (Constant Bit Rate):** A CBR traffic object generates traffic according to a deterministic rate. Packets are of constant size. The OTcl code to implement a CB R traffic source in a simulation is as follows:

```
set my_cbr [new Application/Traffic/CBR]
$my_cbr attach-agent $udp
$ns at <time> "$my_cbr start"
```

Parameters:

- start: starts sending packets according to the configuration parameters
- stop: stops sending packets

Configuration parameters:

- PacketSize_: constant size of packets generated e.g 48
- rate_: sending rate e.g. 64kb
- interval_: (optional) interval time between packets e.g 0.05
- random_: Flag to introduce noise in the departure times; default is off, 1 for on
- maxpkts_: the maximum number of packets to send e.g 1000

2. **Exponential:** Traffic is determined by an exponential distribution. Packets are a constant size. This produces an on/off distribution. Packets are sent at a fixed rate during on periods. No packets are sent during off periods.

TCL code to implement an exponential traffic source in a simulation:

```
set my_exp [new Application/Traffic/Exponential]
```

Configuration parameters

- PacketSize_: constant size of packets generated e.g 210
- burst_time_: average on time for the generator e.g. 500ms
- idle_time_: average off time for the generator e.g 500ms
- rate_: sending rate during the "on" time e.g. 100k

3. **Pareto:** The distribution for traffic generation is taken from a pareto on/off distribution. This is generally used to generate aggregate traffic that exhibits long range dependency. The following is OTcl code to implement a Pareto traffic source in a simulation. Idle times are taken from a pareto distribution.

set my_pareto [new Application/Traffic/Pareto]

Configuration parameters

PacketSize_: constant size of packets generated e.g. 210

burst_time_: average on time for the generator e.g. 500ms

- idle_time_: average off time for the generator e.g. 500ms
- rate_: sending rate during the "on" time e.g. 100k
- shape_: the shape parameter used by the pareto distribution e.g. 1.5
-

4. •**TrafficTrace:** Traffic is generated according to a trace file. The binary file must contain 2 x 32 fields in network (big-endian) byte order. The first field contains the time in ms until next packet is generated. The second field contains the length in bytes of the next packet. The method filename of the Tracefile class associates a trace file with the Tracefile object.

The following is OTcl code to implement a Trace file traffic source in a simulation:

```
set t_file [new Tracefile]

$t_file filename <file>
set src [ new Application/Traffic/Trace]
$src attach-tracefile $t_file
```

where $t_file is a binary file and the two fields in the file contain inter-packets times in milliseconds and packet size in bytes.

**More on TCP Traffic Applications:**

Two simulation applications exist to send traffic on top of a TCP object: Application/FTP and Application/Telnet.

1. **File Transfer protocol** (FTP- for simulating bulk data transfer)

OTCL Code for using FTP in a simulation:

```
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ns at <time> "$ftp start"
```

Parameters:

- attach-agent: attach-agent: attaches an Application/FTP agent to an agent
- start: start the Application/FTP to send data
- stop: stop sending data
- produce n: where n is the counter of packets to be sent
- producemore n: where n is the new increased value of packets to be sent
- send n: similar to producemore, but sends n bytes instead of packets

## 2. Telnet

OTcl code for using Telnet in a simulation:

```
set telnet [new Application/Telnet]

$telnet attach-agent $tcp
```

Parameters

- start: start producing packets
- stop: stop producing packets
- attach-agent: attaches a Telnet object to an agent

## Trace File

Below 3 lines is a part of a trace file.

```
-----------------------------------
+ 1.825127 2 3 tcp 1040 ---A--- 1 1.0 3.1 30 303
- 1.825367 2 3 cbr 210          0 0.0 3.0 203 274
r 1.82558 3 2 ack 40          1 3.1 1.0 29 302
-----------------------------------
```

Each line shows the information of a packet. We will explain the meaning of each column.

**Column 1** The name of event

+: Packet enter to a Queue

-: Packet exit a Queue

r: Packet is received by a node

d: Packet is drop

**Column 2** Time for event in second[sec]

**Column 3, 4** Position of event (Link). Number in column 3, 4 is the number of node the event is occurred when data send from node number in column 3 to node number in column 4.

**Column 5** The type of Packet
Cbr: Packet created by CBR Application, sent by UDP
Agent Tcp: Packet created by FTP Application, sent by
TCP Agent Ack: Packet created by TCPSink Agent (ACK)

**Column 6** Packet size [bytes].

**Column 7** Flag: using for first retransmit packet in retransmission control A is Flag value. Actually, the flag isn't shown for all retransmission packet.

**Column 8** Flow number which packet belong to( refer to 3.2.3).

**Column 9** Packet sender. For example, if the value is 1.0, it means that node which have ID equal 1 send data from port 0.

**Column 10** Packet receiver. For example, if the value is 3.1, it means that node which has ID equal 3 receive data by port 1

**What is a port?**

In realistic network, a computer have to receive data from many service (Web, Mail...). In NS2, there are many Agent will be set. For this reason, if we only record the ID of sender and receiver, we will not know the packet belong to which Agent. Using different port for each Agent is a way to solve this problem. Actually, in relistic network, individual port is used for each service.

**Column 11** The sequence number of transmit packet

**Column 12** The ID number of that Packet. Each packet have a individual ID number.

## Objective: Using Trace File and Plotting using AWK scripts and Xgraph- Trace Analysis

In last lab, we have created nodes, links, applied traffic and studied about trace file. In this lab, we will be using that trace file along with AWK scripts to analyze our network.

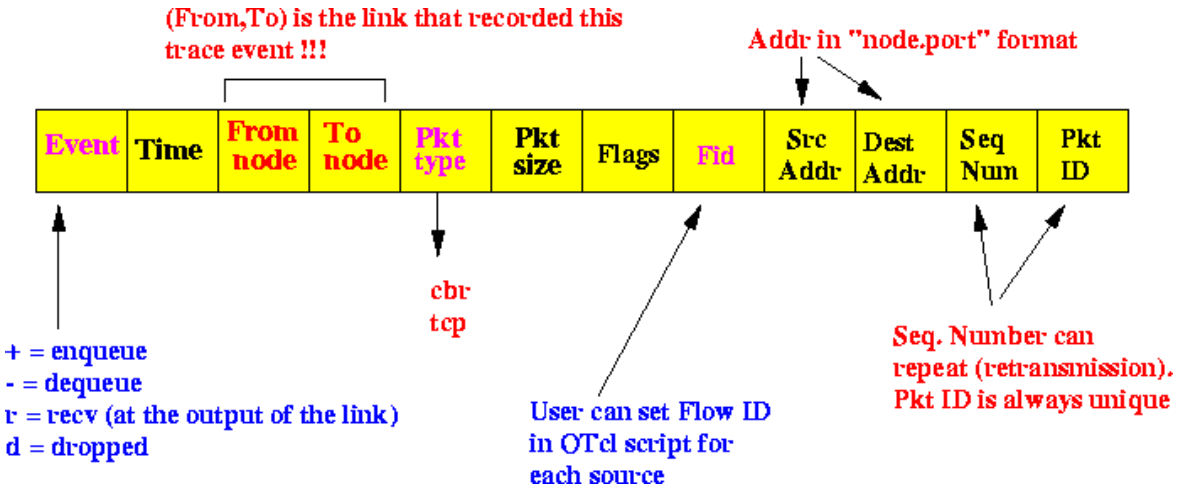
Analysis of a network:

- Delay Throughput
- No of Packets dropped, enqueued, dequeued. No of
- packets received by a particular node. Etc
-

For analyzing the network , we will be using :

1. AWK Scripts
2. Xgraph
3. Trace Files

## Trace File Revisited:




## AWK Scripts:

AWK is a high level programming language which is used to process text files, named after its three orginal author's name:

**A** : Alfred **A**ho

**W** : Peter **W**einberger **K**
: Brian **K**ernighan

AWK Scripts are very good in processing the data from the log (trace files) which we get from NS2.

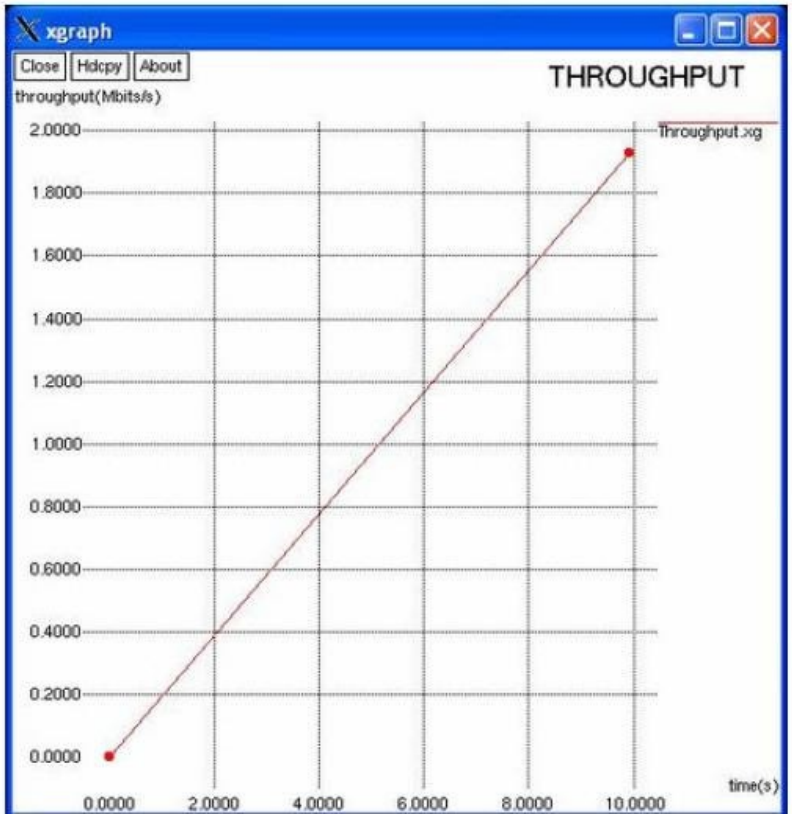
### AWK PROGRAM STRUCTURE

Awk program structure contains mainly three parts;

1. *Begin*
2. *Content*
3. *End*

```
BEGIN {<initialization>}

<pattern1> {<actionSet1>}

<pattern2> {<actionSet2>}

. . .

END {<final finalActionSet>}
```

**BEGIN** : Begin deals with what to be executed prior to text file processing, normally




which is used to initialize variable values or constants.

**CONTENT** : Script which process the text file. In this part, AWK moves lines by lines (i.e., records by records) and executes the <actionSet> if the current line match with the pattern. The actions repeats until AWK reaches the end of the file.

**END** : This part explains what to be executed after the text file processing ie. what to print on the terminal or to show output in terminal.

**Saving the awk file** : delay.awk

## Running AWK script

On command prompt write the following to save the result in a separate text file awk -f

delay.awk out.tr > filename.txt

**XGRAPH**

Xgraph is a plotting program which can be used to create graphic representations of simulation results. You can create output files in your Tcl scripts, which can be used as data sets for xgraph. Call xgraph to display the results with the command “xgraph <data-

file>”. Below you can see a screenshot of an xgraph window.

To run Xgraph from a shell prompt
xgraph file.xg **or** ./xgraph file.txt

or

xgraph out1.txt -t "x-axis" -x "Time" -y "y-axis" or

To plot data contained in multiple files, use xgraph

out1.txt out2.txt out3.txt

**STEPS TO BE FOLLOWED:**

1. Create the network topology. ( create nam and tr files)
2. Attach the agents, application.
3. Run the simulation.
4. Write AWK file.
5. Run AWK file.
6. Save the result of AWK file in a separate text file
7. Plot the graph using Xgraph.

This is the **sample AWK script for delay calculation**

```
BEGIN { highest_packet_id=0; }
{
        action = $1; time = $2;
        from = $3; to = $4; type
        = $5; pktsize = $6;
        flow_id = $8; src = $9;
        dst = $10; seq_no = $11;
        packet_id = $12;

        if ( packet_id > highest_packet_id ) highest_packet_id = packet_id; if
        ( start_time[packet_id] == 0 )
        start_time[packet_id] = time; if(
        action == "r" ) {
                end_time[packet_id] = time;
```

```
        } else {
                end_time[packet_id] = -1;
        }
}
END {
for(packet_id=0; packet_id < highest_packet_id; packet_id++) { start =
        start_time[packet_id];
        end =end_time[packet_id];
        packet_duration = end-start;
        if( start < end )
        printf("%f %f\n",start, packet_duration);
}}
```

"$1" stands for column 1 value in the trace file that we will be referring to. $2 to 2nd column of the trace file and so on.

Another example AWK file to calculate number of packets dropped / received etc.

```
BEGIN {

number_of_drops=0;
number_of_received=0;
number_of_enqued=0;
number_of_dequeue=0;
}
{
parameter1=$1
paramter2=$2
parameter3=$3
parameter4=$4
parameter5=$5
parameter6=$6
parameter8=$8
parameter9=$9
parameter10=$10
parameter11=$11
parameter12=$12
if (parameter1=="r")
number_of_received++;
if(paramter1=="-")
```

```
number_of_dequeue++;
if(parameter1=="d") number_of_drops+
+; if(parameter1=="+")
number_of_enqued++;
} END {
printf("number of received:%d,number of dequeue:
%d",number_of_received,number_of_dequeue); printf("\nnumber of drops:
%d,number of enqued:%d",number_of_drops,number_of_enqued);
}
```

**Exercise:**

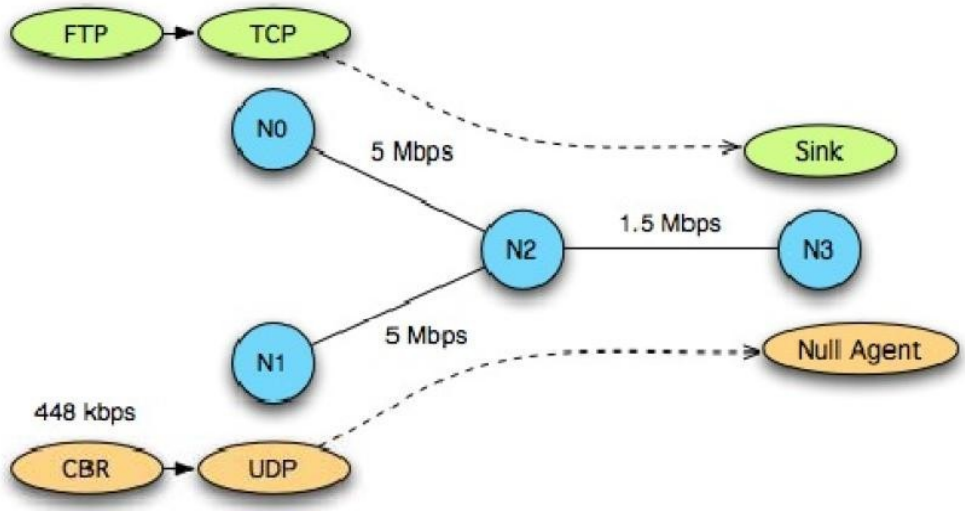
1. Simulate the Lab Assignment-1 Q1 scenario with node 1 and node 3 as senders and node5 as receiver. Attach TCP agent to node 1 and CBR agent with node3. Schedule the sending of data at different time events. Record all the events in a trace file.

2. Simulate the Lab Assignment-1 Q2 scenario with node 1 as sender and node 6 as receiver. Run the FTP application on node 1 and analyze the output.

3. Simulate the Lab Assignment-1 Q2 for TELNET traffic.

4. Write a program in which there are six nodes and node (i) is connected with a duplex link to node (i+1). Create the ping agents and attach to any of the two nodes. Simulate the transmission of ping message over these two nodes.

5. Create a network as below



Now analyze the trace file and do the following

- Count the total number of received packets at node 2 and node 3 Count
- total number of drop packets at node 2
- Calculate the throughput of all the links.

Also plot the graph for comparing the throughput of all the links.

**Q6**. Design a network topology that should contain at least one intermediate node. Use both TCP and UDP protocols with FTP and CBR traffic generated, respectively. Make sure that there is a congested link (bottleneck link).

- Calculate the total throughput of that link, with respect to time and plot the graph.
- Calculate the throughput of that link with respect to time, only for TCP packets and plot the graph.
- Calculate the throughput of that link with respect to time, only for UDP packets and plot it.
- Calculate and plot the average delay.

6. Design a program to simulate a star topology with four edge nodes (nodes N1 to N4) and a single core node (node N5). Each edge node is connected to the core node with a full duplex link, propagation delay is 10 ms for each link. Three CBR-over-UDP sources are attached to each edge node, so that each edge node sends packets to every other edge node. Each source generates 200 pkts per sec and packets have different size depending on the destination node. Sources also start sending packets at different times, depending on the destination. Packet sizes and transmission start times for each destination are as follows:

| Destination | Size (bytes) | Start time |
|---|---|---|
| 1 | 100 | 0.2 |
| 2 | 200 | 0.4 |
| 3 | 300 | 0.6 |
| 4 | 400 | 0.8 |

Bandwidth on each link must be set to the value that guarantees maximum link utilization and no packet losses. Justify your choice of the link bandwidth.